



TECHNICAL DEEP-DIVE

Editing PDFs in Prisma Scan

How the **upload-a-PDF** and **on-page editing** (text boxes + white-out) features are built — the flow, the coordinate math, the code, and the interview questions that go with them.

Flutter · Riverpod

file_picker

pdfx (render)

Syncfusion PDF (write)

inline TextField overlay

1 What we built

① Upload a PDF

Import any existing `.pdf` from the device, store it, and open it in the reader — every page viewable, then editable.

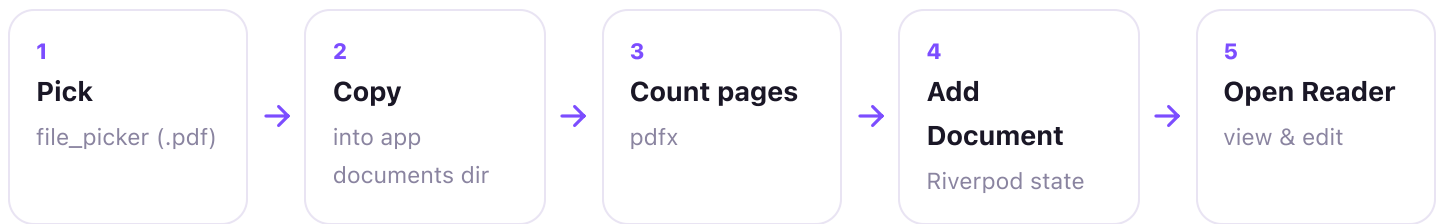
② On-page editor

Add **text boxes** you type into directly on the page, and **white-out** rectangles to cover old text. Move, resize, delete, then flatten to a new PDF.

Honest scope: this overlays new content — it does not rewrite a PDF's original embedded text (and scanned pages have no text underneath, only an image). See §4 for why, and the OCR path forward.

2 Feature 1 — Upload a PDF

A small, reliable flow that reuses the existing reader/editor.



The core code

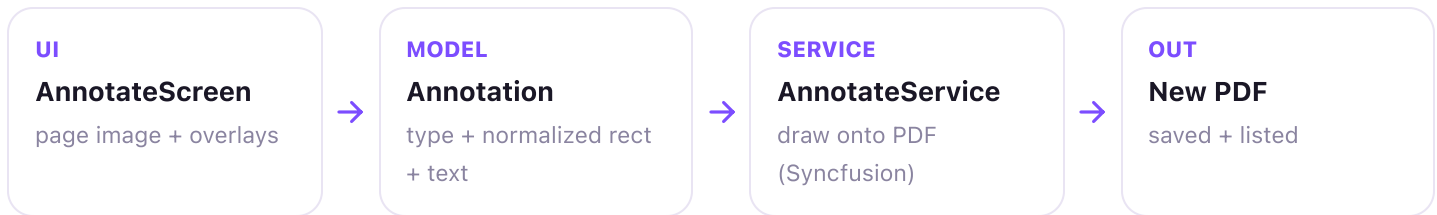
```
// import_service.dart – pick a PDF and copy it into app storage
Future<File?> pickPdf() async {
  final files = await FilePicker.pickFiles(
    type: FileType.custom, allowedExtensions: ['pdf']);
  if (files.isEmpty) return null; // user cancelled
  final dir = await getApplicationDocumentsDirectory();
  return File(files.first.path!).copy('${dir.path}/scans/${files.first.name}');
}
```

Gotcha we hit: `file_picker` requires iOS 15+, so we bumped the iOS deployment target (Podfile + Xcode project). A one-line native-config change, easy to miss.

3 Feature 2 — On-page editor

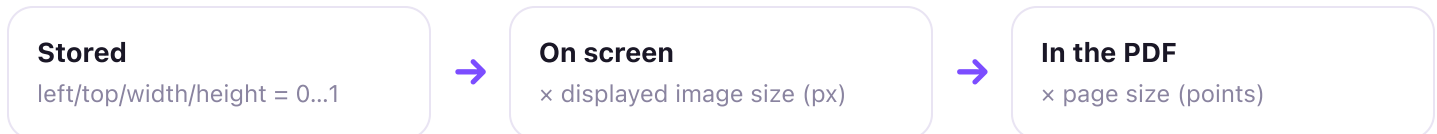
Three pieces: a **model**, a **screen** that overlays widgets on a rendered page, and a **service** that flattens them onto the PDF.

Architecture



The key idea: normalized coordinates

An annotation is stored in **page-relative units from 0 to 1** (top-left origin) — *not* screen pixels. This is the trick that makes everything line up, because the same numbers map cleanly to both the on-screen preview and the final PDF.



Because Syncfusion's drawing origin is also top-left, exporting is a plain multiply — no flipping or guessing:

```
// annotate_service.dart – flatten each overlay onto the page
final size = page.getClientSize(); // page size in points
final rect = Rect.fromLTWH(
  a.left * size.width, a.top * size.height, // 0..1 → points
  a.width * size.width, a.height * size.height);
if (a.type == AnnoType.whiteout) {
  page.graphics.drawRectangle(brush: PdfSolidBrush(white), bounds: rect);
} else {
  page.graphics.drawString(a.text, PdfStandardFont(helvetica, a.fontSize),
    brush: PdfSolidBrush(ink), bounds: rect);
}
```

Inline (WYSIWYG) text — not a dialog

The first version popped a dialog to type. We replaced it: each selected text box is a real `TextField` positioned on the page, so you type **exactly where the text will appear**. To keep a focused text box still draggable, we added a **move handle** (top-left) and a **resize handle** (bottom-right).

```
// a selected text annotation renders as an editable field, in place
editing
? TextField(controller: _controller, autofocus: true, maxLines: null,
  onChanged: (v) => a.text = v,
  style: TextStyle(fontSize: a.fontSize * dh / pagePth), // scale pts->px
  decoration: const InputDecoration.collapsed(hintText: 'Type...'))
: Text(a.text); // static when not selected
```

- **Move / resize / delete** — drag handles update the same 0..1 numbers; a trash button in the top bar removes the selected overlay.
- **Per-page** — annotations are kept in a `Map<pageIndex, List<Annotation>>`; the page image is rendered by `pdfx` and cached.
- **Off the UI thread** — the same pattern as scanning: heavy PDF work never blocks the screen.

4 Why "edit the real text" is overlay, not replace

The single most misunderstood thing about PDFs — worth knowing for interviews.

- A PDF stores text as **glyphs pinned to x/y coordinates**, with **no paragraphs and no reflow** — change a word's length and it overlaps the next.
- Fonts are often **embedded & subsetted**, so retyping needs the exact same font data.
- A **scanned** page has **no text at all** — it's an image; there is nothing to "edit" until OCR adds a text layer.

Approach	Effort	Notes
Overlay + white-out (what we did)	Medium	Covers ~95% of real needs; free with Syncfusion.
OCR → extract text	Medium	Makes scans searchable/selectable; a new doc, not the original layout.
True in-place text edit	Very hard	Realistic only via a paid SDK (Apyrse / PSPDFKit / Foxit).

Next step: add on-device **OCR** (`google_mlkit_text_recognition`, free) so scanned pages become selectable/extractable text — the honest way to "edit words" on a scan.

5 Interview questions & answers

Q How do you edit an existing PDF in a Flutter app?

Render pages with `pdfx` for display, and write with `syncfusion_flutter_pdf`. To edit, I overlay Flutter widgets (text fields, rectangles) on a rendered page image, then on export I load the PDF with Syncfusion and draw those overlays onto each page's graphics before saving a new file.

Q How do you keep an overlay aligned between the phone screen and the final PDF?

Store every annotation in **normalized coordinates (0...1, top-left origin)** relative to the page — not pixels. On screen I multiply by the displayed image size; on export I multiply by the page size in points. Syncfusion's drawing origin is also top-left, so the mapping is a straight multiply with no coordinate flip.

Q Why not let users edit the actual text of the PDF?

PDF text is positioned glyphs with no reflow, fonts are often subsetting, and scanned pages have no text layer at all. True in-place editing needs a commercial SDK. The pragmatic, free solution is overlay + white-out (cover old text, type new), plus OCR for scans.

Q You have a popup vs inline editing — which and why?

Inline. A dialog is a context switch and hides the page. I render the selected text annotation as an in-place `TextField` so users type exactly where the text lands (WYSIWYG). The trade-off is gesture conflict: a focused field consumes drags, so I added an explicit move handle to keep it draggable.

Q How do you handle the file after import — permissions, storage?

`file_picker` returns a temp path; I copy it into the app's documents directory (`path_provider`) so it persists and is sandbox-safe, then store the path on the `Document` model. On iOS this needed bumping the deployment target to 15 for the plugin.

Q How would you make this testable?

The export logic is a pure-ish service (`AnnotateService.apply`) that takes a source path + annotations and returns a file — unit-testable with a fixture PDF. Services are injected via Riverpod providers, so the UI can be widget-tested with a fake service. Coordinate mapping is pure math and trivially unit-tested.

Q Where could performance be a problem, and how do you avoid it?

Rendering pages and building PDFs is heavy. Pages are rendered once and cached per index; the scan→PDF path runs pixel work in a background isolate via `compute()` . For very large PDFs I'd lazy-render pages and paginate rather than render all upfront.